Pattern Iterator

Pattern Itérateur en français

Sommaire

- Introduction au Design Pattern Iterator
- Application du Pattern avec les interfaces IEnumerator et IEnumerable de C#
- Application du Pattern avec des interfaces génériques
- Conclusion

Introduction au Design Pattern Iterator

C'EST QUOI

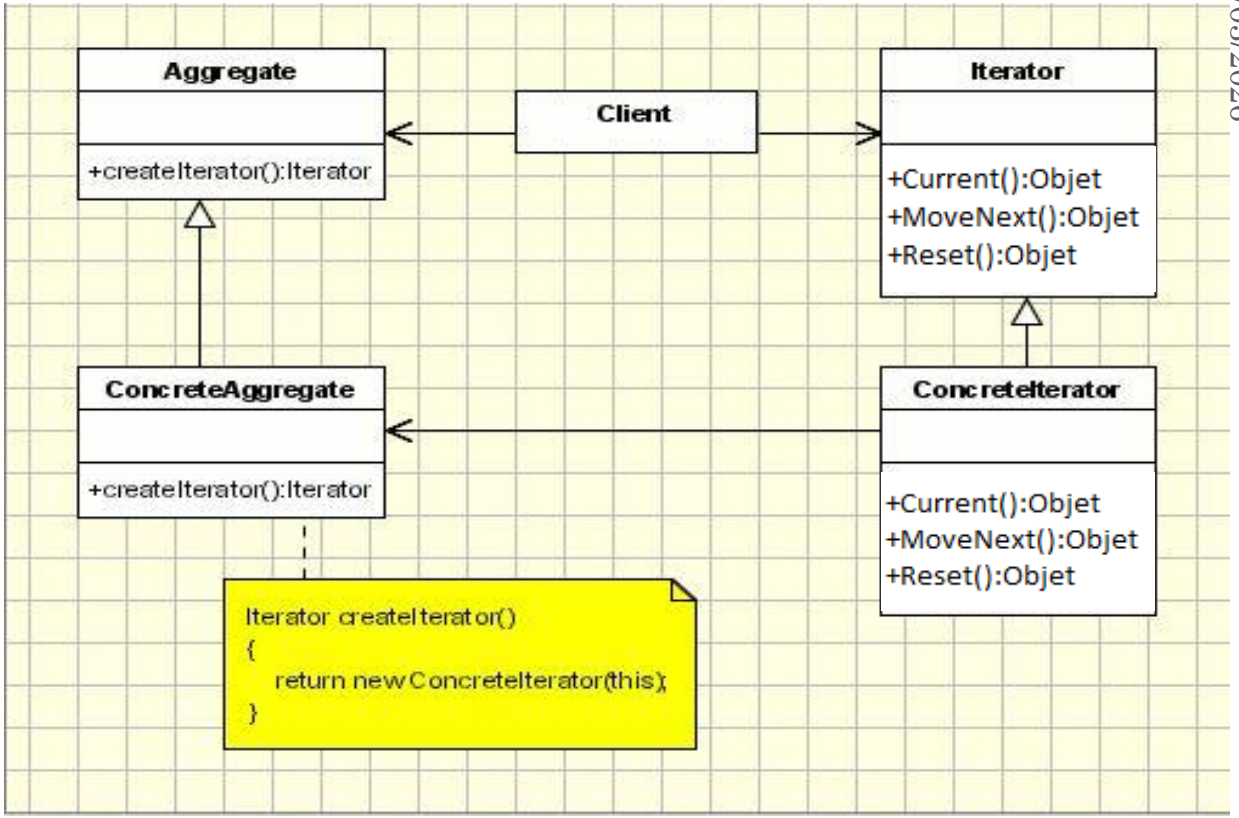
- Le pattern **Iterator** est un patron de conception (*design pattern*) comportemental. Ce patron permet de parcourir tous les éléments contenus dans une collection.
- Un Itérateur ressemble à un pointeur disposant essentiellement de deux fonctions simples:
  - *accéder* à l'élément pointé en cours (dans la collection)
  - *se déplacer* pour pointer vers l'élément suivant.
- Il faut pouvoir créer un Itérateur pointant sur le premier élément ; ainsi que déterminer à tout moment si l'Itérateur a épuisé la totalité des éléments du conteneur.

Quand l'utiliser

- Le but d'un Itérateur est de permettre à son utilisateur de *parcourir* le conteneur, c'est-à-dire d'accéder séquentiellement à tous ses éléments pour leur appliquer un traitement, en cachant la complexité de parcourir les éléments de la collection.
- La collection peut stocker les éléments de la façon qu‘elle veut, tout en permettant à l'utilisateur de la traiter comme une simple liste.
- Le plus souvent l’itérateur est conçu en même temps que la classe-conteneur qu'il devra parcourir, et ce sera le conteneur lui-même qui créera et distribuera les itérateurs pour accéder à ses éléments.

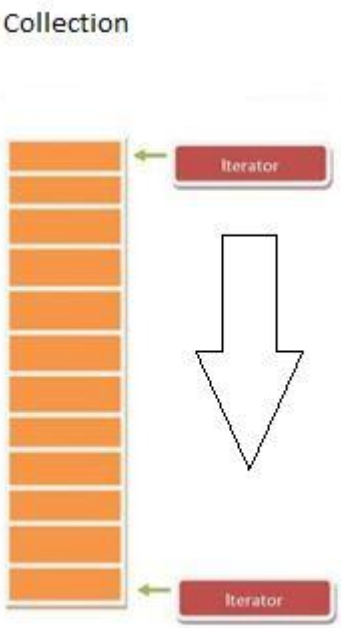
Pattern Itérator

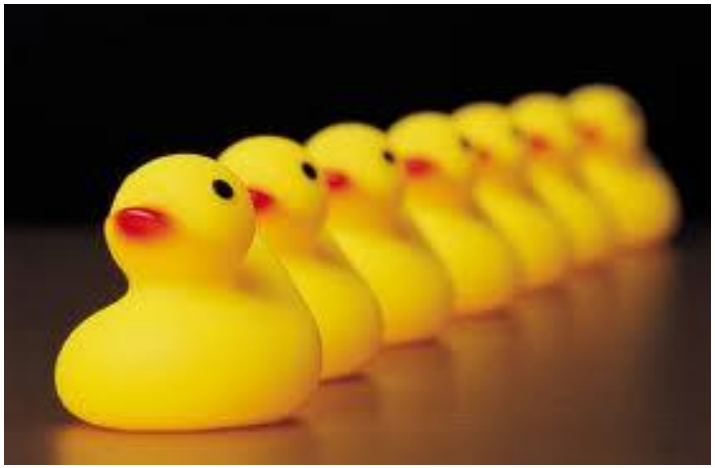
Dark Iterator



- **IIterator** - Cette interface représente le AbstractIterator, la définition de l'itérateur
- **BookIterator** - Il s'agit de la mise en œuvre de Iterator (implémente l'interface IIterator)
- **IContainer** - C'est une interface définissant le agréger
- **BooksCollection** - Une mise en œuvre de la collection

Diagramme de classe



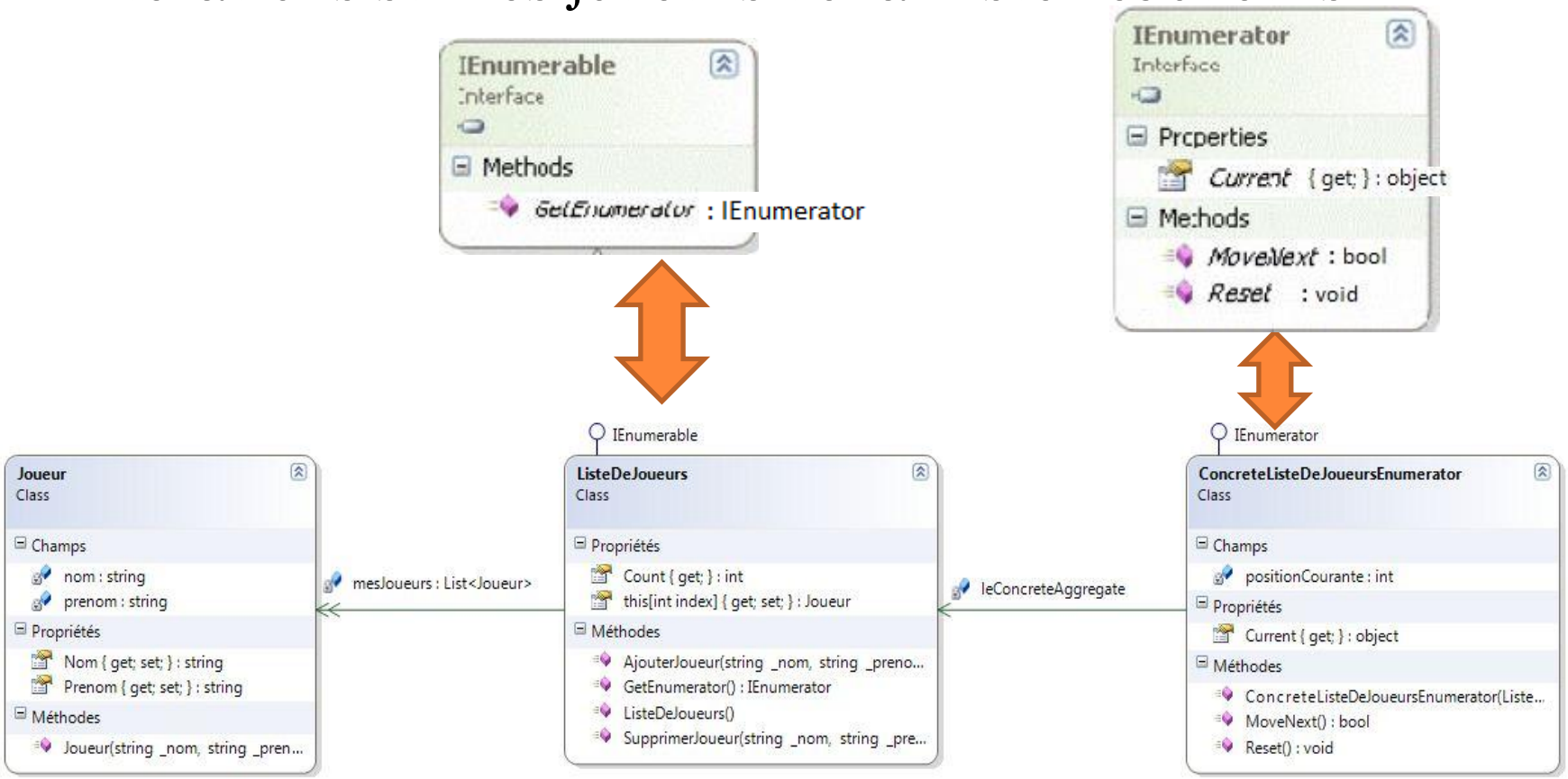


collection de canards

Application du Pattern avec les interfaces IEnumerator et IEnumerable de c#

Exemples : Liste de joueurs

- Diagramme de classe permettant de réaliser des itérations sur les joueurs de la ListeDeJoueurs



Pattern Iterator en C#

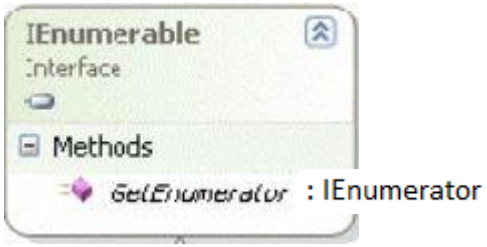
- Interface IEnumerator

```
namespace System.Collections
{
    ...public interface IEnumerator
    {
        // Résumé :
        //     Obtient l'élément actuel dans la collection.
        //
        // Retourne :
        //     Élément actuel dans la collection.
        //
        // Exceptions :
        //   System.InvalidOperationException:
        //     L'énumérateur précède le premier élément ou suit le dernier élément de la
        //     collection.
        object Current { get; }

        // Résumé :
        //     Avance l'énumérateur à l'élément suivant de la collection.
        //
        // Retourne :
        //     true si l'énumérateur a pu avancer jusqu'à l'élément suivant ; false si l'énumérateur
        //     a dépassé la fin de la collection.
        //
        // Exceptions :
        //   System.InvalidOperationException:
        //     La collection a été modifiée après la création de l'énumérateur.
        bool MoveNext();
        //
        // Résumé :
        //     Rétablit l'énumérateur à sa position initiale, qui précède le premier élément
        //     de la collection.
        //
        // Exceptions :
        //   System.InvalidOperationException:
        //     La collection a été modifiée après la création de l'énumérateur.
        void Reset();
    }
}
```

Pattern Iterator en C# (suite)

- Interface IEnumerable




```
using System.Runtime.InteropServices;

namespace System.Collections
{
    // Résumé :
    //     Expose l'énumérateur, qui prend en charge une itération simple sur une collection
    //     non générique.
    [ComVisible(true)]
    [Guid("496B0ABE-CDEE-11d3-88E8-00902754C43A")]
    public interface IEnumerable
    {
        // Résumé :
        //     Retourne un énumérateur qui itère au sein d'une collection.
        //
        // Retourne :
        //     Objet System.Collections.IEnumerator pouvant être utilisé pour itérer au
        //     sein de la collection.
        [DispId(-4)]
        IEnumerator GetEnumerator();
    }
}
```

1ère façon de parcourir: créer une méthode permettant d'accéder à un élément de la collection

- GetJoueurALaPosition de la classe ListeDeJoueur:

```
public Joueur GetJoueurALaPosition(int position)
{
    return mesJoueurs[position];
}
```

- Code du parcours de la ListeDeJoueur (dans le code p représente la listeDeJoueur)

```
for (int i = 0; i < p.Count; i++)
{
    Joueur j = p.GetJoueurALaPosition(i);
    Console.WriteLine(j.Nom + " , " + j.Prenom);
}
```

2ème façon de parcourir: Surcharge de l'opérateur [ ]

- Dans la philosophie C#, on utilise pour accéder à un élément d'une collection les [ ] (crochets)

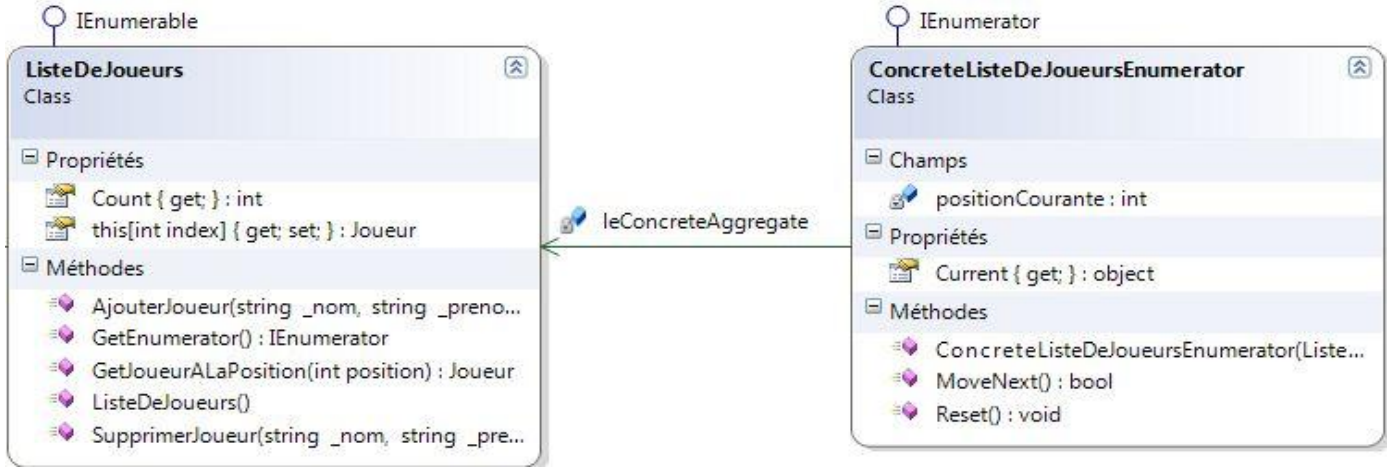
```
for (int i = 0; i < p.Count;i++ )
{
    Joueur j = p[i];
    Console.WriteLine(j.Nom + " , " + j.Prenom);
}
```

- On doit surcharger avec le code suivant dans la classe ListeDeJoueur:

```
public Joueur this[int index]
{
    get { return mesJoueurs[index]; }
    set { mesJoueurs.Insert(index, value); }
}
```

3ème façon de parcourir: utilisation du pattern Iterator

- Cette façon fonctionne pour n'importe quel itérateur



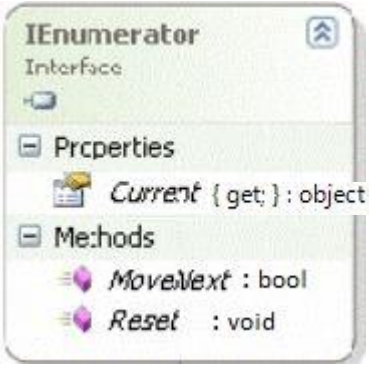
- Exemple utilisant l'itérateur implémenté:

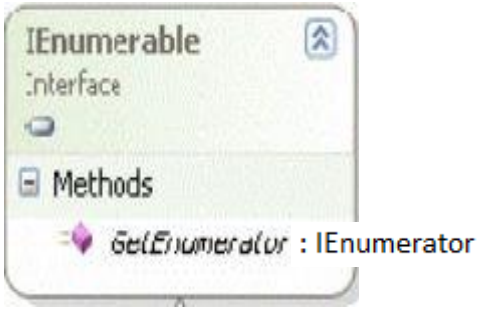
```
ConcreteListeDeJoueursEnumerator c = (ConcreteListeDeJoueursEnumerator)p.GetEnumerator();

while (c.MoveNext())
{
    Joueur j = (Joueur)c.Current;
    Console.WriteLine(j.Nom + " , " + j.Prenom);
}
```

4ème façon: utilisation du pattern Iterator du Framework C#

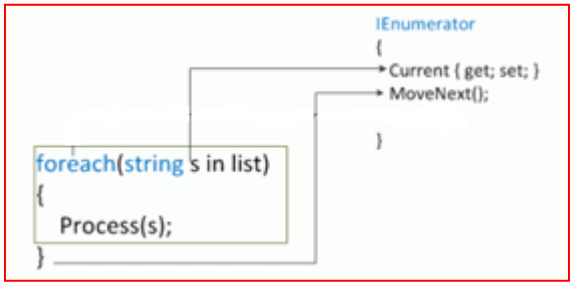
- Le code est possible grâce à l'implémentation des interfaces ***IEnumerator & IEnumerable*** en ***C#***





- Le code est utilisé de manière transparente !

```
foreach (Joueur j in p)
{
    Console.WriteLine(j.Nom + " , " + j.Prenom);
}
```



Application du Pattern avec des interfaces génériques

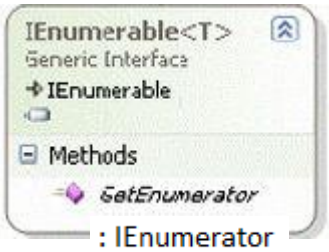
Des interfaces qui utilisent des génériques < T >

Les interfaces IEnumerator<T> et IEnumerable<T> sont des interfaces typées pour des Joueur dans une liste typées

```
...public interface IEnumerator<out T> : IDisposable, IEnumerator
{
    // Résumé :
    //     Obtient l'élément de la collection situé à la position actuelle de l'énumérateur.
    //
    // Retourne :
    //     Élément dans la collection à la position actuelle de l'énumérateur.
    T Current { get; }
}
```

```
public class ConcreteListeDeJoueursEnumerator: IEnumerator<Joueur>
```

```
...public interface IEnumerable<out T> : IEnumerable
{
    // Résumé :
    //     Retourne un énumérateur qui itère au sein de la collection.
    //
    // Retourne :
    //     System.Collections.Generic.IEnumerator<T> pouvant être utilisé pour itérer
    //     au sein de la collection.
    IEnumerator<T> GetEnumerator();
}
```




```
public class ListeDeJoueurs:IEnumerable<Joueur>
```

Réalisation des interfaces

Exemple de réalisation d'une implémentation pour la liste de joueur

1. Le **current** renvoie un Joueur et non plus un Object

```
public Joueur Current
{
    get
    {
        if (positionCourante == -1)
        {
            throw new InvalidOperationException();
        }
        else
        {
            return leConcreteAggregate[positionCourante];
        }
    }
}
```

Réalisation des interface(suite)

2. Réalisation de l'interface IEnumerable générique

```
public IEnumerator<Joueur> GetEnumerator()
{
    return new ConcreteListeDeJoueursEnumerator(this);
}
```

3. Réalisation de l'interface IEnumerable

```
System.Collections.IEnumerator System.Collections.IEnumerable.GetEnumerator()
{
    return GetEnumerator();
}
```

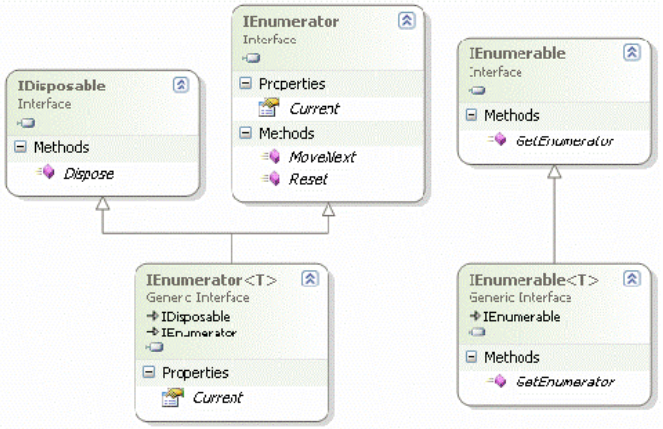
La méthode *dispose* de *IDisposable*

- L'interface IEnumerator réalise l'interface IDisposable et elle s'appelle au moment de la destruction de l'Objet *Enumerateur* et dans cette méthode libère explicitement les ressources non managées de l'instance d'Enumerateur (un fichier, une fenêtre, …) grâce au ***Garbage Collector*** **.**

```
public void Dispose()
{
    GC.SuppressFinalize(this);
}
```

- Le Garbage Collector libère automatiquement la mémoire allouée à un objet managé lorsque cet objet n'est plus utilisé.

L'INTERFACE *IDISPOSABLE*




```
using System.Runtime.InteropServices;

namespace System
{
    ...public interface IDisposable
    {
        // Résumé :
        //     Exécute les tâches définies par l'application associées à la libération ou
        //     à la redéfinition des ressources non managées.
        void Dispose();
    }
}
```

Les objets ne peuvent pas être explicitement détruits. Le ramasse-miettes s'occupe de libérer la mémoire lorsqu'il n'existe plus aucune référence pointant sur un objet. Toutefois , pour les objets gérant des types non managés, il est possible d'implémenter l'interface ***IDisposable*** pour spécifier des traitement à effectuer au moment de la libération de la ressource.

Quand on appelle une classe qui implémente l'interface IDisposable il faut utiliser **« Try/Finally »** afin de s'assurer que les ressources non managées sont éliminés même si une exception interrompt l'application.

Ressources :http://msdn.microsoft.com/en-us/library/system.idisposable.aspx
http://codingly.com/2009/04/01/petit-rappel-sur-le-disposing-pattern-en-c/

FIN